NAME:P.MOSHIYA

REG NO:192572142

CO5_ASSESSMENT 1

CONSTRAINT -BASED PROBLEM SOLVING

1. Real-Time Face Recognition for Campus Security

A university wants to deploy a camera-based security system at the entrance that can identify registered students and staff in real time.
The system should:

- Capture faces from live CCTV/video streams.
- Detect faces under different lighting conditions.
- Extract suitable facial features.
- Recognize registered individuals.
- Maintain real-time performance on a low-cost computer.
- Minimize false recognition.

Design a complete OpenCV-based face recognition pipeline from image acquisition to final identification. Explain the preprocessing, face detection, feature extraction, recognition, and optimization techniques used. Justify how your system achieves accuracy and real-time performance.

2. Intelligent Traffic Violation Detection System

A traffic department wants to automatically monitor vehicles using roadside cameras.
The system should:

- Detect vehicles from live video.
- Track vehicles across consecutive frames.
- Identify vehicles crossing a predefined line.
- Count vehicles accurately.
- Detect abnormal or unwanted movements.
- Display vehicle count and alerts in real time.

Design an OpenCV-based solution incorporating video acquisition, preprocessing, object detection, motion analysis, tracking, and visualization. Explain how you would handle shadows, camera noise, dynamic backgrounds, and multiple vehicles to minimize counting errors.

3. Automated Document Processing System

A company receives thousands of scanned documents that may contain blurred text, uneven illumination, noise, and different font sizes.
The system should:

- Automatically improve document quality.
- Remove noise and unwanted background.

- Convert the document into a suitable format for OCR.
- Segment text regions.
- Extract readable text efficiently.
- Process a large number of documents in batch mode.

Design an OpenCV-based image processing pipeline for this system. Explain the role of grayscale conversion, thresholding, filtering, morphological operations, segmentation, and geometric correction. Justify how each stage improves OCR accuracy.

4. Real-Time Multi-Person Tracking and Attendance System

A college wants to automatically record student attendance using classroom CCTV cameras.
The system should:

- Detect multiple faces simultaneously.
- Track each person across video frames.
- Recognize registered students.
- Avoid repeatedly marking the same student.
- Handle slight head movement and temporary face disappearance.
- Operate continuously with minimum processing delay.

Design a complete OpenCV-based system using face detection, feature extraction, recognition, and tracking. Explain how you maintain identity across frames and prevent false or duplicate attendance records.

5. Smart Home Surveillance and Intrusion Detection

A home security system uses a fixed camera to monitor an entrance continuously.
The system should:

- Detect significant motion.
- Distinguish actual movement from camera noise and small background changes.
- Track moving objects.
- Identify suspicious activity.
- Generate an alert when an unauthorized person enters a predefined region.
- Operate in real time with limited computational resources.

Design an OpenCV-based surveillance pipeline using background subtraction, filtering, morphological operations, contour/feature detection, motion analysis, and tracking. Explain how your approach reduces false alarms caused by shadows, illumination changes, trees, and other dynamic background objects.

1. Real-Time Face Recognition for Campus Security

Problem Statement

A university needs a real-time security system that can identify registered students and staff from CCTV cameras. The system must work under different lighting conditions, minimize false recognition, and run efficiently on a low-cost computer.

Proposed OpenCV-Based Pipeline

Camera/Video Acquisition → Preprocessing → Face Detection → Face Alignment → Feature Extraction → Face Recognition → Confidence Checking → Identification → Logging/Alert

1. Image Acquisition

The system uses an entrance CCTV camera or webcam as the input source.

OpenCV can capture video using:

cap = cv2.VideoCapture(0)

For CCTV, the camera stream or video URL can be supplied instead of 0.

Each frame is continuously captured and processed.

2. Preprocessing

The captured frame may contain noise, poor illumination, and different face orientations.

The following preprocessing techniques can be applied:

- Convert the frame from BGR to grayscale.
- Resize the frame to reduce processing time.
- Apply Gaussian filtering to reduce noise.
- Improve brightness using histogram equalization or CLAHE.
- Normalize the face region before recognition.

For example:

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

gray = cv2.equalizeHist(gray)

CLAHE can be preferred when lighting is uneven because it improves local contrast without excessively amplifying noise.

3. Face Detection

OpenCV Haar Cascade or DNN-based face detection can be used.

For a lightweight system, Haar Cascade is suitable:

```
faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(40, 40)
)
```

The detector identifies the coordinates of every face in the frame.

4. Face Alignment

A detected face may be tilted or slightly rotated.

Face alignment makes the eyes, nose, and mouth approximately occupy consistent positions.

This improves recognition because the recognition algorithm receives faces with similar orientation.

5. Feature Extraction

Instead of comparing complete images directly, important facial characteristics are extracted.

Features may include:

- Eye position
- Nose structure
- Mouth structure
- Facial shape
- Distances between facial landmarks
- Deep facial embeddings

For a basic OpenCV implementation, the detected and normalized face can be converted into a feature representation.

For a more accurate practical system, a pretrained face-recognition/DNN model can generate a numerical embedding for each face.

6. Face Database

Before deployment, registered students and staff are enrolled.

Example:

Face Database

├── Student_001

├── Student_002

├── Student_003

├── Staff_001

└── Staff_002

Each person's facial features are extracted and stored with their ID.

7. Recognition

The feature vector obtained from the live camera is compared with the stored feature vectors.

A similarity measure such as Euclidean distance or cosine similarity can be used.

Conceptually:

Live Face

↓

Feature Vector

↓

Compare with Database

↓

Nearest Matching Identity

↓

Confidence Check

If the distance is below the selected threshold, the person is recognized.

Otherwise:

Unknown Person

8. Minimizing False Recognition

False recognition is an important constraint.

The following methods can be used:

- Use a suitable recognition threshold.
- Require multiple consecutive frames to agree.
- Reject low-quality or very small faces.
- Use face alignment.
- Maintain multiple training images for each person.
- Use different lighting conditions during registration.
- Use high-quality facial embeddings.
- Do not identify a person from a single weak detection.
- Mark a person as recognized only when confidence remains high.

For example, the system may require the same identity to be detected in several consecutive frames before displaying the final result.

9. Real-Time Optimization

A low-cost computer has limited CPU/GPU resources.

Optimization techniques include:

- Resize the input frame.
- Process every second or third frame instead of every frame.

- Detect faces periodically.
- Track detected faces between detection frames.
- Use lightweight Haar Cascade or optimized DNN models.
- Avoid unnecessary image conversions.
- Store face features instead of repeatedly processing registration images.
- Use multithreading when appropriate.

10. Final Output

The system displays:

Student_001

Confidence: High

Status: Registered

or

UNKNOWN PERSON

Security Alert

A rectangle can be drawn around the detected face:

cv2.rectangle(frame, (x,y), (x+w,y+h), (0,255,0), 2)

Overall Working

CCTV Camera

↓

Capture Frame

↓

Resize + Noise Reduction

↓

Lighting Correction

↓

Face Detection

↓

Face Alignment

↓

Feature Extraction

↓

Compare with Registered Database

↓

Confidence/Threshold Check

↓

Recognized / Unknown

↓

Display + Log + Alert

Justification

The proposed system provides accuracy through illumination correction, face alignment, suitable feature extraction, threshold-based recognition, and multiple-frame verification. Real-time performance is achieved by resizing frames, using lightweight detection, detecting faces periodically, and tracking them between detections. Therefore, the system is suitable for continuous campus entrance monitoring on a relatively low-cost computer.

2. Intelligent Traffic Violation Detection System

Problem Statement

A traffic department needs an automated system that detects vehicles, tracks them, counts vehicles crossing a predefined line, identifies abnormal movement, and generates real-time alerts.

Proposed OpenCV Pipeline

Video Acquisition → Preprocessing → Background/Motion Analysis → Vehicle Detection → Tracking → Line Crossing Detection → Movement Analysis → Counting → Alert → Visualization

1. Video Acquisition

A roadside CCTV camera continuously captures the traffic scene.

cap = cv2.VideoCapture("traffic.mp4")

For a live camera:

cap = cv2.VideoCapture(0)

Each frame is processed continuously.

2. Preprocessing

Traffic video can contain camera noise and unnecessary details.

Preprocessing includes:

- Frame resizing
- Grayscale conversion
- Gaussian blur
- Noise reduction
- Contrast enhancement when required

Example:

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray, (5,5), 0)

3. Vehicle Detection

Vehicles can be detected using:

- Background subtraction
- Contour detection

- Haar-based methods
- OpenCV DNN object detection
- Pretrained object detection models

For a fixed camera, background subtraction can be effective.

OpenCV provides:

```
background = cv2.createBackgroundSubtractorMOG2()

fgmask = background.apply(frame)
```

The foreground mask identifies moving objects.

4. Morphological Processing

The foreground mask may contain small holes and noise.

Morphological operations improve the mask.

```
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))


mask = cv2.morphologyEx(
    fgmask,
    cv2.MORPH_OPEN,
    kernel
)


mask = cv2.morphologyEx(
    mask,
    cv2.MORPH_CLOSE,
    kernel
)
```

Opening removes small noise.

Closing fills small gaps inside vehicle regions.

5. Contour Detection

Contours are extracted from the foreground mask.

```
contours, _ = cv2.findContours(
    mask,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)
```

Very small contours are ignored.

A bounding box is generated around each valid vehicle.

6. Vehicle Tracking

Detection alone cannot determine whether the same vehicle appears in consecutive frames.

A tracker assigns an ID to every detected vehicle.

Example:

Frame 1 → Vehicle 1

Frame 2 → Vehicle 1

Frame 3 → Vehicle 1

Frame 4 → Vehicle 1

Tracking methods can include:

- Centroid tracking
- CSRT
- KCF
- MOSSE

- SORT-style tracking

For multiple vehicles, each object maintains:

Vehicle ID

Center Point

Bounding Box

Previous Position

Current Position

7. Predefined Line Crossing

A virtual line is drawn across the road.

Example:

---------- LINE ----------

Vehicle movement →

The centroid of each vehicle is calculated.

If:

previous_y < line_y

and later:

current_y >= line_y

the vehicle is considered to have crossed the line.

The vehicle ID is stored in a set so it is counted only once.

8. Vehicle Counting

Example:

Vehicle 1 → Crossed → Count = 1

Vehicle 2 → Crossed → Count = 2

Vehicle 3 → Crossed → Count = 3

Using vehicle IDs prevents the same vehicle from being counted repeatedly.

9. Abnormal Movement Detection

The system can analyze vehicle trajectories.

Possible abnormal movements include:

- Moving in the wrong direction
- Sudden stopping
- Illegal lane change
- U-turn in a restricted area
- Entering a restricted region
- Excessive movement outside the permitted lane

For example, if vehicles are expected to move from left to right and a tracked vehicle moves right to left, the system can generate:

ALERT: WRONG-WAY MOVEMENT

10. Handling Shadows

Shadows can be incorrectly detected as vehicles.

To reduce this problem:

- Use shadow suppression in background subtraction.
- Apply morphological filtering.
- Analyze contour size and shape.
- Compare object dimensions across frames.
- Use object classification when higher accuracy is required.

11. Handling Camera Noise

Camera noise can produce small false foreground regions.

Gaussian or median filtering can be used.

Small contours are removed using an area threshold:

```
if cv2.contourArea(contour) > 500:
    # valid object
```

12. Dynamic Background

Trees, rain, waving objects, and changing illumination can create false motion.

Solutions include:

- MOG2 background subtraction.
- Adaptive background modeling.
- Morphological filtering.
- Region-of-interest restriction.
- Object-size filtering.
- Combining motion detection with object classification.

13. Handling Multiple Vehicles

Every detected vehicle receives a unique ID.

Vehicle 1 → ID 1

Vehicle 2 → ID 2

Vehicle 3 → ID 3

Tracking maintains these IDs across frames.

This prevents duplicate counting and allows individual movement analysis.

14. Visualization

The output displays:

Vehicle Count: 24

Wrong-Way Alerts: 2

Bounding boxes and vehicle IDs are drawn on the video.

Overall Working

CCTV Video

↓

Frame Acquisition

↓

Resize + Noise Reduction

↓

Background Subtraction / Object Detection

↓

Morphological Filtering

↓

Contour/Object Detection

↓

Vehicle Tracking

↓

Centroid Calculation

↓

Line Crossing Detection

↓

Vehicle Counting

↓

Movement Analysis

↓

Alerts + Visualization

Justification

The combination of object detection, tracking, centroid analysis, and virtual line crossing provides accurate vehicle counting. Morphological operations, area filtering, shadow handling, and adaptive background subtraction reduce false detections. Tracking ensures that the same vehicle is not counted repeatedly. Therefore, the proposed system can operate continuously and provide real-time traffic monitoring and violation alerts.

3. Automated Document Processing System

Problem Statement

A company receives a large number of scanned documents containing blur, noise, uneven illumination, different font sizes, and unwanted backgrounds. The system must improve the documents before OCR.

Proposed OpenCV Pipeline

Document Input → Grayscale Conversion → Noise Reduction → Illumination Correction → Thresholding → Morphological Processing → Text Segmentation → Geometric Correction → OCR → Text Storage

1. Image Acquisition

Scanned documents or document images are loaded into the system.

image = cv2.imread("document.jpg")

For batch processing, a folder containing thousands of documents can be processed automatically.

2. Grayscale Conversion

Color information is generally unnecessary for standard printed document OCR.

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

This reduces the image from three color channels to one intensity channel.

Benefits:

- Reduces computational requirements.
- Simplifies thresholding.
- Makes text/background separation easier.

3. Noise Reduction

Scanned documents may contain:

- Dust
- Salt-and-pepper noise
- Scanner artifacts
- Small unwanted dots

Median or Gaussian filtering can be applied.

```
blur = cv2.GaussianBlur(gray, (5,5), 0)
```

For salt-and-pepper noise, median filtering can be useful.

4. Uneven Illumination Correction

Uneven lighting causes some parts of the document to appear darker than others.

CLAHE can improve local contrast:

```
clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8,8)
)


enhanced = clahe.apply(gray)
```

This makes characters more distinguishable from the background.

5. Thresholding

Thresholding converts the grayscale document into a binary image.

A fixed threshold can be used for clean documents:

```
_, binary = cv2.threshold(
    enhanced,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

Otsu's method automatically selects a suitable threshold.

For uneven illumination, adaptive thresholding can be used:

```
binary = cv2.adaptiveThreshold(
    enhanced,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)
```

6. Morphological Operations

Morphological operations improve the binary document.

Opening:

Removes small noise and isolated dots.

Closing:

Connects broken parts of characters and fills small gaps.

Example:

```
kernel = cv2.getStructuringElement(
    cv2.MORPH_RECT,
    (3,3)
)

clean = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    kernel
)
```

7. Text Segmentation

The system identifies different text regions.

Possible methods include:

- Contour detection
- Connected component analysis
- Projection profiles
- Morphological grouping

Contours can identify blocks containing text.

The document can be divided into:

```
Document
│
├── Header
├── Paragraph 1
├── Paragraph 2
```

├── Table

└── Footer

8. Geometric Correction

Scanned documents may be tilted.

This is called **skew**.

The system can estimate the document/text orientation and rotate it to make the text horizontal.

Perspective correction can also be applied when the document was photographed at an angle.

This is important because OCR engines work better when text is properly aligned.

9. OCR Conversion

After preprocessing, the resulting image is supplied to an OCR engine such as Tesseract or another OCR system.

Conceptually:

Clean Binary Image

↓

Text Segmentation

↓

OCR

↓

Recognized Text

10. Different Font Sizes

The OCR input can be normalized by:

- Resizing small text regions.
- Maintaining sufficient image resolution.
- Processing text regions independently.

- Avoiding excessive image compression.

11. Batch Processing

Since thousands of documents must be processed, the system automatically reads files from a directory.

Documents/

document1.jpg

document2.jpg

document3.jpg

...

Each document passes through the same pipeline.

The output can be stored as:

Output/

document1.txt

document2.txt

document3.txt

12. Improving OCR Accuracy

Stage	**Purpose**
Grayscale	Removes unnecessary color information
Filtering	Removes noise
CLAHE	Improves local contrast
Thresholding	Separates text from background
Opening	Removes small noise
Closing	Connects broken text regions

Stage	Purpose
Segmentation	Separates text regions
Deskewing	Corrects document orientation
Perspective correction	Corrects camera angle
Resizing	Improves recognition of small text

Overall Working

Scanned Document

↓

Image Loading

↓

Grayscale Conversion

↓

Noise Filtering

↓

Contrast / Illumination Correction

↓

Thresholding

↓

Morphological Processing

↓

Text Region Segmentation

↓

Deskew / Perspective Correction

↓

OCR

↓

Readable Text

↓

Batch Storage

Justification

Each stage prepares the document for the next stage. Noise filtering removes unwanted pixels, thresholding separates characters from the background, morphological operations repair text regions, and geometric correction aligns the document. These preprocessing operations significantly improve the quality of the image supplied to OCR. Batch processing allows thousands of documents to be handled automatically and efficiently.

4. Real-Time Multi-Person Tracking and Attendance System

Problem Statement

A college wants to automatically record attendance using classroom CCTV. The system must detect multiple faces, maintain identity across frames, recognize registered students, avoid duplicate attendance, and work continuously with minimum delay.

Proposed OpenCV Pipeline

CCTV → Frame Acquisition → Preprocessing → Multi-Face Detection → Face Alignment → Feature Extraction → Recognition → Multi-Object Tracking → Identity Association → Attendance Validation → Database → Display

1. Video Acquisition

The classroom CCTV continuously provides frames.

cap = cv2.VideoCapture("classroom.mp4")

For a live camera:

cap = cv2.VideoCapture(0)

2. Preprocessing

Each frame is prepared using:

- Resizing
- Grayscale conversion where appropriate
- Noise reduction
- Contrast enhancement

The frame can be resized to reduce computational cost.

3. Multi-Face Detection

The face detector scans the complete classroom frame and identifies all visible faces.

Example:

```
faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5
)
```

Output:

Face 1 → Student A

Face 2 → Student B

Face 3 → Student C

Face 4 → Unknown

4. Face Alignment

Each detected face is normalized.

Alignment compensates for:

- Slight head rotation
- Different face positions
- Small variations in pose

This improves recognition consistency.

5. Feature Extraction

A feature representation is created for every detected face.

The system can use facial embeddings generated by a suitable recognition model.

For example:

Student A → Feature Vector A

Student B → Feature Vector B

Student C → Feature Vector C

6. Face Recognition

Each live feature vector is compared with the registered student database.

The system determines the closest match and checks the confidence threshold.

Matched → Student Name

No reliable match → Unknown

7. Multi-Person Tracking

Detection is not required on every frame.

Once a face/person is detected, a tracker can follow it across subsequent frames.

For example:

Frame 1 → Student A → ID 1

Frame 2 → ID 1

Frame 3 → ID 1

Frame 4 → ID 1

This reduces repeated detection and improves speed.

8. Maintaining Identity

Every person receives a unique tracking ID.

The system stores:

Tracking ID

Student Name

Last Position

Last Seen Time

Recognition Confidence

If the face moves slightly, the tracking ID remains associated with the same person.

9. Temporary Face Disappearance

A student's face may temporarily disappear because of:

- Head movement
- Turning sideways
- Occlusion
- Another student passing in front

The tracker can maintain the person's identity for a limited number of frames.

When the face becomes visible again, the system can associate the new detection with the existing track.

10. Preventing Duplicate Attendance

A set/database table can store students already marked present.

Example:

Present Students:

Student A

Student B

Student C

If Student A appears in another frame, the system checks:

Is Student A already marked?

↓

YES

↓

Do not mark again

Thus, attendance is recorded only once per session.

11. False Recognition Prevention

To reduce false attendance:

- Use a recognition confidence threshold.
- Require multiple successful recognitions.
- Ignore extremely small faces.
- Reject low-quality face images.
- Use several registration images per student.
- Use tracking consistency.
- Mark attendance only after identity remains stable for a specified period.

For example:

Frame 1 → Student A

Frame 2 → Student A

Frame 3 → Student A

Frame 4 → Student A

Stable Identity → Mark Attendance

12. Real-Time Optimization

To reduce processing delay:

- Resize frames.
- Detect faces periodically.
- Track between detections.
- Process only the classroom ROI.
- Use lightweight detection.
- Store precomputed student features.
- Avoid repeated recognition of already confirmed students.

13. Attendance Database

A simple attendance record can contain:

Student ID | Name | Date | Time | Status

Example:

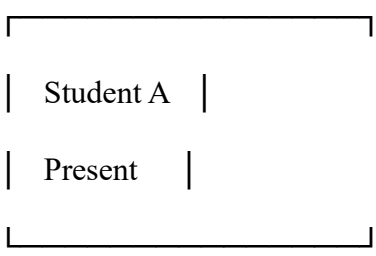
101 | Student A | 24-08-2026 | 09:05 | Present

102 | Student B | 24-08-2026 | 09:06 | Present

14. Visualization

The system displays a bounding box around each recognized person.

Example:



The screen may also show:

Total Detected: 35

Present: 32

Unknown: 3

Overall Working

Classroom CCTV

↓

Frame Acquisition

↓

Preprocessing

↓

Multi-Face Detection

↓

Face Alignment

↓

Feature Extraction

↓

Face Recognition

↓

Tracking

↓

Identity Association

↓

Stable Identity Verification

↓

Attendance Database Check

↓

Mark Once

↓

Display Attendance

Justification

The system combines face recognition with tracking rather than performing independent recognition on every frame. Tracking maintains identity during small movements and temporary face disappearance, while a database/set prevents duplicate attendance. Multiple-frame verification and confidence thresholds reduce false attendance. Frame resizing and periodic detection maintain real-time performance.

5. Smart Home Surveillance and Intrusion Detection

Problem Statement

A home uses a fixed camera to continuously monitor an entrance. The system should detect meaningful movement, track objects, identify suspicious activity, and generate an alert when an unauthorized person enters a predefined region.

Proposed OpenCV Pipeline

Camera → Frame Acquisition → Preprocessing → Background Subtraction → Noise Removal → Shadow Suppression → Morphological Processing → Contour Detection → Object Tracking → ROI Analysis → Suspicious Activity Detection → Alert

1. Video Acquisition

The fixed camera continuously captures the entrance.

cap = cv2.VideoCapture(0)

The system reads each frame continuously.

2. Preprocessing

The input video may contain sensor noise and small variations.

Preprocessing includes:

- Frame resizing
- Gaussian/median filtering
- Grayscale conversion
- Contrast adjustment when required

Example:

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray, (5,5), 0)
```

3. Background Subtraction

Because the camera is fixed, the background can be modeled.

OpenCV provides MOG2:

```
backSub = cv2.createBackgroundSubtractorMOG2(
    history=500,
    varThreshold=50,
    detectShadows=True
)


fgmask = backSub.apply(frame)
```

The foreground mask identifies regions that differ from the background.

4. Shadow Handling

Shadows can be detected as foreground even though no actual person has entered.

MOG2 provides shadow detection.

The system can suppress shadow pixels before further processing.

Other techniques include:

- Brightness comparison.
- Color analysis.
- Contour shape analysis.
- Object-size filtering.

5. Morphological Operations

The foreground mask may contain small holes and noise.

Opening removes isolated noise.

Closing fills small gaps.

```
kernel = cv2.getStructuringElement(
    cv2.MORPH_ELLIPSE,
    (5,5)
)


mask = cv2.morphologyEx(
    fgmask,
    cv2.MORPH_OPEN,
    kernel
)


mask = cv2.morphologyEx(
    mask,
    cv2.MORPH_CLOSE,
    kernel
```

)

6. Significant Motion Detection

Small movements should not trigger an alarm.

For example:

Small leaf movement → Ignore

Small camera noise → Ignore

Large human-sized region → Consider

The contour area is checked.

```
if cv2.contourArea(contour) > MIN_AREA:
    # significant movement
```

7. Contour Detection

Contours are obtained from the cleaned foreground mask.

```
contours, _ = cv2.findContours(
    mask,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)
```

Bounding boxes are created around significant objects.

8. Object Tracking

A detected moving object receives an ID.

Person → Track ID 1

Person → Track ID 1

Person → Track ID 1

Tracking helps determine whether the object is continuously moving toward the entrance.

Possible OpenCV-compatible tracking methods include:

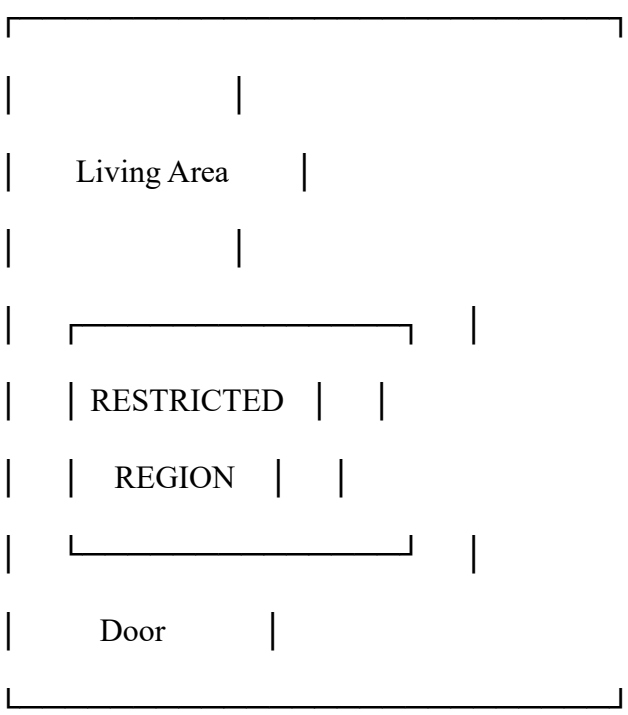
- Centroid tracking
- CSRT
- KCF
- MOSSE

9. Predefined Region of Interest

A restricted area is defined near the entrance.

Example:

Camera View



If a tracked person enters this region, the system evaluates it as a possible intrusion.

10. Suspicious Activity Detection

Suspicious activity can be identified using:

- Entry into restricted ROI.
- Movement during restricted hours.

- Long-duration presence.
- Repeated movement near the entrance.
- Movement in an unexpected direction.
- An object crossing a virtual line.

Example:

Person detected

↓

Enters restricted ROI

↓

Check authorization/time condition

↓

Unauthorized

↓

ALERT

11. Reducing False Alarms

Camera Noise

Use Gaussian or median filtering and remove small contours.

Shadows

Use shadow detection and brightness/color analysis.

Illumination Changes

Use adaptive background modeling and avoid triggering an alarm from a sudden global brightness change.

Trees and Moving Background Objects

Use:

- Region of interest.
- Minimum object size.
- Morphological filtering.
- Object tracking.
- Persistence across multiple frames.

A single-frame detection should not immediately trigger an alarm.

12. Multi-Frame Verification

A robust system verifies movement over several frames.

Example:

Frame 1 → Motion detected

Frame 2 → Motion detected

Frame 3 → Large object detected

Frame 4 → Object enters ROI

Frame 5 → Object remains in ROI

↓

Confirmed Intrusion

This prevents temporary noise from generating false alarms.

13. Real-Time Optimization

Because the system may run on a low-cost computer:

- Resize frames.
- Process a predefined ROI.
- Use lightweight background subtraction.

- Track objects rather than repeatedly detecting them.
- Process only necessary frames.
- Remove very small contours early.
- Avoid unnecessary image operations.

14. Alert Generation

When an intrusion is confirmed, the system can display:

⚠ INTRUSION ALERT

Unauthorized movement detected

A timestamp can also be recorded.

Date: 24-08-2026

Time: 20:15:32

Event: Unauthorized Entry

Overall Working

Fixed Security Camera

↓

Frame Acquisition

↓

Resize + Noise Filtering

↓

Background Subtraction

↓

Shadow Suppression

↓

Morphological Processing

↓

Contour Detection

↓

Object Filtering

↓

Object Tracking

↓

Restricted ROI Analysis

↓

Multi-Frame Verification

↓

Suspicious Activity Detection

↓

Intrusion Alert

Justification

The fixed-camera environment makes background subtraction suitable for detecting movement. Filtering, morphological operations, contour-area thresholds, and shadow suppression reduce false detections. Tracking maintains object identity across frames, while a predefined ROI and multi-frame verification ensure that only meaningful movement triggers an alert. Frame resizing, ROI processing, and lightweight algorithms allow the system to operate in real time on limited hardware.